Exercise 2

In this exercise, we implemented a visualization pipeline using Python and VTK. We used the `head.vti` file to create a 3D visualization showing the bones and the skin of a human head.

Prerequisites

First, we need to make sure the VTK library is installed to run this notebook. The installation command is provided below.

In [ ]:
```
# Install the VTK package required for this assignment
!pip install vtk
```

Settings and Imports

Here, we imported the necessary VTK modules. Since we are using a Jupyter Notebook environment, passing command-line arguments (`-b` or `-a`) is not practical. Therefore, we adapted the required command-line parameters into boolean variables:

- `show_bbox = False` corresponds to not using the `-b true` flag, so the bounding box is hidden by default.
- `enable_animation = True` corresponds to the `-a true` flag for the bonus task.

These variables will control our visualization logic in the final step.

In [14]:
```
import vtkmodules.vtkRenderingOpenGL2
from vtkmodules.vtkIOXML import vtkXMLImageDataReader
from vtkmodules.vtkFiltersModeling import vtkOutlineFilter
from vtkmodules.vtkCommonDataModel import vtkPlane
from vtkmodules.vtkFiltersCore import vtkContourFilter
from vtkmodules.vtkInteractionStyle import vtkInteractorStyleTrackballCamera
from vtkmodules.vtkRenderingCore import (
    vtkActor,
    vtkPolyDataMapper,
    vtkRenderWindow,
    vtkRenderWindowInteractor,
    vtkRenderer,
)

# Settings
show_bbox = False
enable_animation = True
input_filename = 'head.vti'

def read_input_file(filename):
    reader = vtkXMLImageDataReader()
    reader.SetFileName(filename)
    reader.Update()
    return reader.GetOutput()
```

```
data = read_input_file(input_filename)
```

Task 1: Bounding Box

For Task 1, we created a bounding box for the input data to visualize the spatial extent of the volume. We used the `vtkOutlineFilter` which generates lines mapping the boundaries of the data. Then, we mapped it and assigned it to an actor, setting the color of the bounding box to green as requested.

In [15]:
```
def task1(input_data):
    # Create the outline filter
    outline = vtkOutlineFilter()
    outline.SetInputData(input_data)

    # Create the mapper
    mapper = vtkPolyDataMapper()
    mapper.SetInputConnection(outline.GetOutputPort())

    # Create the bounding box actor
    outActor = vtkActor()
    outActor.SetMapper(mapper)

    # Set the color to green (R=0, G=1, B=0)
    outActor.GetProperty().SetColor(0.0, 1.0, 0.0)

    return outActor

bbox_actor = task1(data)
```

Task 2: Extracting Bones

For Task 2, we extracted the bone structure using `vtkContourFilter`. We first investigated the scalar range of our dataset (0-255). After testing a few different thresholds, we found that an isovalue of `70.0` is the most suitable value for extracting the complete bone structure, including finer details like the jaw and teeth. Finally, we set the actor's color to yellow.

In [16]:
```
def task2(input_data):
    # Create a contour filter for bones
    contour = vtkContourFilter()
    contour.SetInputData(input_data)
    contour.SetValue(0, 70.0)

    # Create the mapper
    mapper = vtkPolyDataMapper()
    mapper.SetInputConnection(contour.GetOutputPort())
    mapper.ScalarVisibilityOff()

    # Create the bones actor
    bonesActor = vtkActor()
    bonesActor.SetMapper(mapper)

    # Set color to yellow (R=1, G=1, B=0)
    bonesActor.GetProperty().SetColor(1.0, 1.0, 0.0)
```

```
    return bonesActor

bones_actor = task2(data)
```

Task 3: Extracting Skin and Clipping

For Task 3, we extracted the skin using `vtkContourFilter` with a lower isovalue of `50.0`. We set the color to red and made it semi-transparent (`Opacity = 0.5`) to see the underlying structures. To match the reference figure, we needed to expose the face. We achieved this by defining a `vtkPlane` with a normal of `(0, 1, 0)`. This cuts the skin along the coronal plane (Y-axis), hiding the front part of the skin and leaving it only on the back of the head like a hood.

In [17]:
```
def task3(input_data):
    # Create a contour filter for the skin
    contour = vtkContourFilter()
    contour.SetInputData(input_data)
    contour.SetValue(0, 50.0)

    # Create a cutting plane
    plane = vtkPlane()

    # Find the center of the data
    bounds = input_data.GetBounds()
    center_x = (bounds[0] + bounds[1]) / 2.0
    center_y = (bounds[2] + bounds[3]) / 2.0
    center_z = (bounds[4] + bounds[5]) / 2.0

    plane.SetOrigin(center_x, center_y, center_z)

    # Cut along the Coronal plane to expose the front face
    plane.SetNormal(0, 1, 0)

    # Create the mapper
    mapper = vtkPolyDataMapper()
    mapper.SetInputConnection(contour.GetOutputPort())
    mapper.ScalarVisibilityOff()
    mapper.AddClippingPlane(plane)

    # Create the skin actor
    skinActor = vtkActor()
    skinActor.SetMapper(mapper)

    # Set color to red (R=1, G=0, B=0) and make it transparent (opacity = 0.5)
    skinActor.GetProperty().SetColor(1.0, 0.0, 0.0)
    skinActor.GetProperty().SetOpacity(0.5)

    return skinActor

skin_actor = task3(data)
```

Final Visualization and Bonus Task

Finally, we combined all the actors into a single visualization window.

- **Task 4**: We implemented a conditional check to append the `bbox_actor` to our render list only if `show_bbox` is True. This fulfills the requirement to hide it by default.
- **Bonus Task (Animation)**: We fulfilled the bonus task by defining a `rotate_camera` function that increments the camera's azimuth. We then hooked this function to the interactor's `TimerEvent` using an observer, and started a repeating timer (every 50ms). This creates a continuous rotation animation in the background.

In [18]:
```
def rotate_camera(obj, event):
    render_window = obj.GetRenderWindow()
    renderer = render_window.GetRenderers().GetFirstRenderer()
    camera = renderer.GetActiveCamera()

    # Rotate the camera by 1 degree
    camera.Azimuth(1)
    render_window.Render()

def show_visualization(actors_list):
    # Create a renderer
    renderer = vtkRenderer()
    renderer.SetBackground(0.1, 0.1, 0.1)

    # Add the Actors to the Renderer
    for actor in actors_list:
        renderer.AddActor(actor)

    # Create Render Window
    window = vtkRenderWindow()
    window.SetSize(800, 600)
    window.AddRenderer(renderer)

    # Create Interactor
    interactor = vtkRenderWindowInteractor()
    interactor.SetRenderWindow(window)
    interactor.SetInteractorStyle(vtkInteractorStyleTrackballCamera())

    # Set the initial camera orientation
    renderer.ResetCamera()
    initial_camera = renderer.GetActiveCamera()
    initial_camera.Azimuth(180)
    initial_camera.SetViewUp(0,0,0)
    initial_camera.Elevation(-90)
    initial_camera.OrthogonalizeViewUp()
    # Initialize interactor before creating timers
    interactor.Initialize()

    # Bonus Task: Animation timer
    if enable_animation:
        interactor.AddObserver('TimerEvent', rotate_camera)
        interactor.CreateRepeatingTimer(50)

    window.Render()
    interactor.Start()

# Task 4: Add actors to the list based on show_bbox
actors_to_show = []
```

```
if show_bbox:
    actors_to_show.append(bbox_actor)

actors_to_show.append(bones_actor)
actors_to_show.append(skin_actor)

show_visualization(actors_to_show)
```